

CAN BOX V0 方案汇总

本文用于冻结本轮"脑子满汉全席"的阶段性结论。\\
当前定位：参考方案，不急于产品化，不提前扩展 V2。\\
后续以 V0 验证为主，需求驱动硬件和软件实现。

一、需求

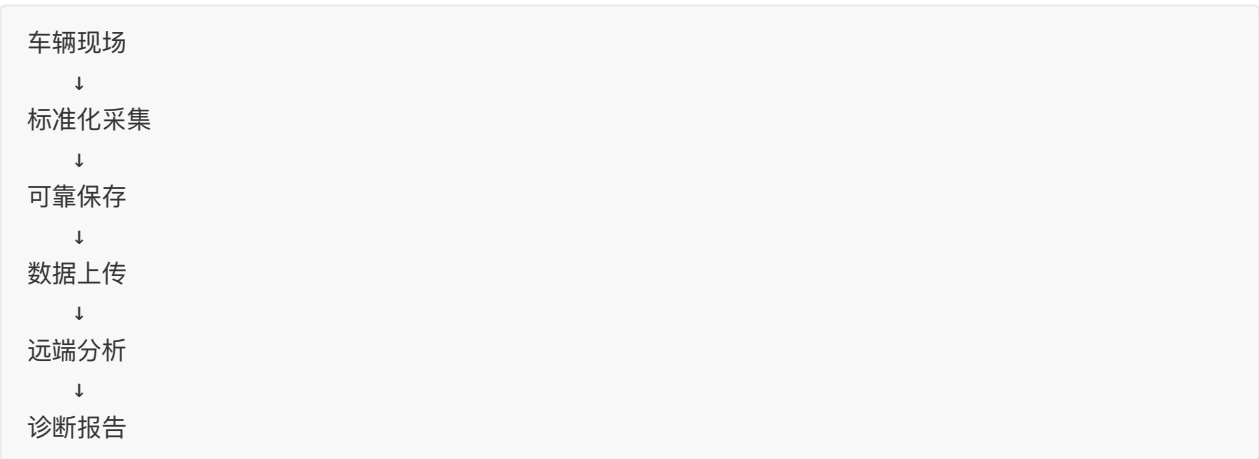
1. 产品目标

CAN BOX 不是通用车载电脑，也不是新的 CAN 硬件诊断仪。

它的目标是：

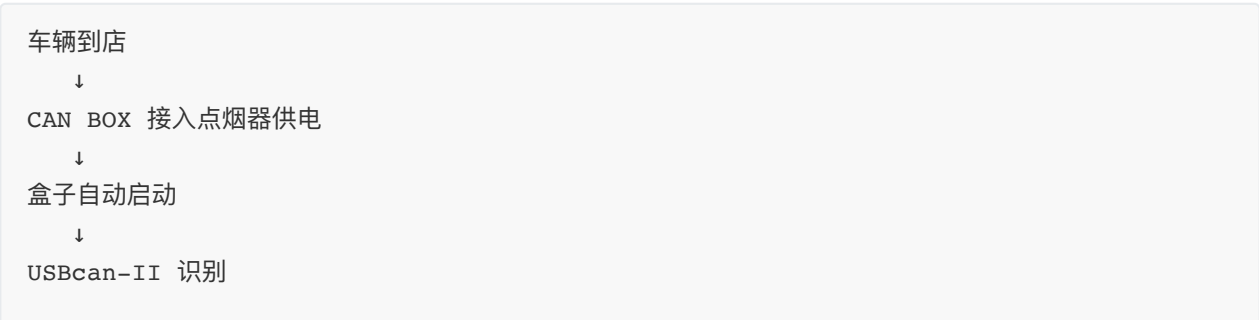
在车辆现场稳定采集 CAN 数据，并按预定义 Case / Script 完成标准化采集，将数据可靠保存和上传，供后端 / Codex 分析并生成诊断报告。

核心链路：



汽修店已有 USBcan-II，因此 **USBcan-II 不属于 CAN BOX BOM**。CAN BOX 的定位是把汽修店已有 CAN 采集硬件变成可联网、可管理、可标准化执行 Case 的 Linux 智能采集网关。

2. 典型现场流程



↓
自动连接手机热点 / 店内 Wi-Fi
↓
手机浏览器打开 Web 管理页
↓
选择车辆 / Case / 工况
↓
开始采集
↓
按 Script 完成操作
↓
结束采集
↓
数据可靠落盘
↓
自动打包 / 上传
↓
后端分析并生成报告

现场使用人员不要求懂 Linux，也不要求理解 CAN 底层。

3. CAN 采集需求

当前以 USBcan-II 为第一版适配对象：

USB Host
↓
USBcan-II
├─ CAN CH1
└─ CAN CH2

需要支持：

- 连续 CAN 采集；
- 准确时间戳；
- 开始 / 停止；
- Case / Session / Round 上下文；
- 原始数据完整保存；
- 滚动文件；
- USBcan 异常识别与恢复；
- 后续允许增加其他 USB CAN Adapter，但 V0 不提前实现。

当前数据量基线：

10 min ≈ 60 MB
30 min ≈ 180 MB / 车
2~3辆 / 天 ≈ 360~540 MB / 天

因此本地存储按 **≥8GB** 作为实用目标。

4. 手机 Web 管理需求

盒子无屏、无键盘，手机承担：

显示
输入
设备管理
网络中转

浏览器至少能够完成：

- 查看设备状态；
- 查看 USBcan 状态；
- 选择 Case；
- 管理 Session；
- 开始 / 停止采集；
- 查看采集时长和文件状态；
- 查看日志；
- 查看存储空间；
- Wi-Fi 配置；
- 查看上传状态；
- 软件升级；
- 重启；
- Recovery。

V0 不要求专用手机 App。

5. 网络需求

Wi-Fi 必须支持：

AP
├ 首次初始化
└ 网络故障恢复

STA
├ 手机热点
└ 汽修店 Wi-Fi

首次启动：

无可⽤已知 Wi-Fi
↓
自动进⼊ AP
↓
广播 CANBOX-XXXX
↓
手机连接
↓
Web 配置网络

正常启动：

上电
↓
寻找已保存 Wi-Fi
↓
自动连接
↓
联网

手机热点可作为 Internet Gateway：

CAN BOX
↓ Wi-Fi
手机热点
↓ 4G / 5G
Internet

因此 V0 不需要独⽴ 4G/5G 模块和 SIM。

6. 数据可靠性

点烟器可能突然断电，因此采集不能依赖正常 shutdown。

要求：

CAN数据
↓
滚动文件
↓
周期 flush / fsync
↓
完成文件确认

突然掉电时，应尽量做到：

最多损失最后⼀个短时间数据⽚段，⽽不是整个 Session。

数据至少关联：

- Device ID;
- Hardware Version;
- Software Version;
- Config Version;
- Case ID;
- Session ID;
- Round;
- 时间;
- 车辆 / VIN（有条件时）。

7. 时间同步

CAN 分析依赖可靠时间轴。

V0 不强制增加硬件 RTC，至少支持：

Internet → NTP

以及：

手机时间

↓

Web

↓

同步给 CAN BOX

8. 设备身份与恢复

每台设备具有唯一身份，例如：

CANBOX-0001

维护 Device ID、MAC、软硬件版本等信息。

恢复能力：

短按 Reset

└ 重启

长按 Recovery

└ 恢复网络配置

↓

自动进入 AP

另外保留 UART 作为开发 / 救砖接口：

UART

|— Boot Log
|— Console
|— Shell
└— Recovery

9. 软件升级

至少需要：

- 查看当前版本；
- 上传升级包；
- 校验；
- 执行升级；
- 记录升级日志；
- 基本失败恢复。

V0 暂不要求完整 A/B 双系统 OTA。

10. 非功能要求

优先级：

稳定性

↓

USB兼容性

↓

数据可靠性

↓

Wi-Fi可靠性

↓

恢复 / 易维护

↓

供货稳定

↓

成本

↓

性能

CAN 当前约 100KB/s 的平均数据量并不要求高性能 CPU。

11. V0 明确不做

当前不要求：

屏幕

触摸

键盘

GPS

蓝牙业务

4G / 5G
Ethernet
内置 CAN
CAN-FD
硬件 RTC
UPS
GPU
NPU / 本地 AI
数据库
复杂云平台
Node.js
专用手机 App

二、硬件方案

1. V0 硬件结构



2. 第一版验证硬件

第一版直接使用现有：

Raspberry Pi 3 Model B Rev 1.2

原因：

- 已有，主控新增成本为 0；
- Linux 成熟；
- USB Host 成熟；
- Wi-Fi 已有；
- TF 存储方便；
- UART / GPIO 已有；
- 非常适合先验证完整软件闭环。

供电第一版：

```
Model 3 点烟器
  ↓
带保险点烟器公头
  ↓
XL4015
12V → 约5.1V
  ↓
短粗 Micro-USB
  ↓
Pi 3B
```

3. 后续低成本主控参考方向

当前较符合需求的方向是 **MT7628 路由器 / IoT Linux 模块**。

典型能力：

```
Linux / OpenWrt
128MB RAM级
板载 Wi-Fi
AP / STA
USB 2.0 Host
UART
GPIO
SPI / I2C
外部存储扩展
```

相比高性能 SBC，它不为 CAN BOX 支付无用的 GPU、HDMI、多核 CPU、大内存等成本。

但 MT7628 只是当前参考候选，**V0 不冻结最终 SoC**。

4. 硬件最低规格

后续选择主控时，入场条件：

```
Linux
+
≥128MB RAM (推荐)
+
USB 2.0 Host
+
Wi-Fi AP / STA
+
≥8GB 数据存储能力
+
UART
```

64MB RAM 理论可做，但 128MB 更适合作为工程下限。

5. 成本参考

USBcan-II 由汽修店提供，不计入 BOM。

当前目标不是追求极限最低价，而是：

百元以内或百元级主机成本已经足够合理，稳定性比再节省十几二十元更重要。

参考目标：

主控模块	¥40~60级目标
存储	¥10~20
电源	¥5~10
PCB / 接口 / 按键	¥10左右
箱体 / 线材	¥10左右
目标 BOM	约 ¥70~100

真正进入批量后再做供应链降本。

6. 未来可能扩展

仅保留可能性，不进入 V0：

- Ethernet；
- 4G/5G；
- USB Device / OTG；
- 第二 USB Host；
- 内置 CAN；
- CAN-FD；
- RTC；
- 更多 GPIO / I²C / SPI 外设。

三、软件方案

1. 总体结构

```
CAN BOX Software
|
├─ OS / BSP
|
├─ canboxd
|
├─ QuickJS
|
├─ uHTTPd / Web
|
└─ System Management
```

核心原则：

Linux 是地基，canboxd 是心脏，QuickJS 是灵活业务逻辑层，uHTTPd/Web 是管理界面，手机是屏幕。

2. OS / BSP

不是开发一个新操作系统，而是最终形成一个专用 Linux Appliance Image。

需要的主要能力：

```
Bootloader
Linux Kernel
RootFS
USB Host
Wi-Fi
Storage
Network
Watchdog
Service Management
```

最终只保留 CAN BOX 所需组件，例如：

```
BusyBox
USB
Wi-Fi
wpa_supplicant
hostapd
dnsmasq
uHTTPd
QuickJS
NTP
canboxd
upgrade
```

不需要：

```
Desktop
X11 / Wayland
Audio
GUI
Python
Node.js
Database
本机编译环境
```

OS 的重点不是把镜像压到极小，而是：

```
稳定启动
USB稳定
Wi-Fi稳定
掉电可靠
自动恢复
升级可靠
```

3. CAN 采集核心：canboxd

建议 CAN 采集核心独立于 Web / QuickJS：

```
canboxd
├──
├── USBcan Adapter
├── Acquisition Engine
├── Timestamp
├── File Writer
├── Rolling File
├── Case Metadata
├── Session
├── Round
├── Device State
└── IPC / Control Interface
```

原则：

```
Web挂了      → CAN继续采
QuickJS挂了  → CAN继续采
Wi-Fi断了    → CAN继续采
uHTTpd重启   → CAN继续采
```

采集可靠性不能依赖管理界面。

后续如果汽修店出现其他 USB CAN 型号，再增加 Adapter Layer；V0 只支持现有 USBcan-II。

4. Web 方案

本轮暂定：

```
uHTTPd
+
Static HTML / CSS / JS
+
QuickJS CGI
```

结构：

```
手机浏览器
  ↓ HTTP
  uHTTPd
  |
  ├── Static Web
  |
  └── CGI / API
      ↓
      QuickJS
      ↓
      canboxd / OS
```

QuickJS 不承担 HTTP Server 本身。

uHTTPd 负责成熟 HTTP 能力，QuickJS 负责：

- Case / Session 业务逻辑；
- 配置；
- 状态转换；
- API 胶水；
- Wi-Fi / 升级等控制逻辑。

这样避免引入 Node.js，也避免使用较重的完整 JS Runtime。

5. Device State

Web 的核心不是复杂 UI，而是统一设备状态。

参考：

```
BOOTING
NETWORK_SETUP
READY
ACQUIRING
STOPPING
PACKING
UPLOADING
UPGRADING
ERROR
```

Web 只负责清楚呈现这些状态并提供对应操作。

6. System Management

包括：

```
Wi-Fi AP / STA
时间同步
Device ID
日志
存储管理
上传 / 重试
Recovery
Watchdog
USBcan 热插拔
软件升级
```

这些是大量"小而确定"的系统胶水工作，非常适合逐项交给 Codex 实现和测试。

四、路线建议

总原则

硬件、OS、应用软件 可以并行。

但必须通过少量固定 Contract 解耦，避免互相等待。

```
CAN BOX
|
├─ A. 硬件线
├─ B. OS / BSP线
└─ C. 应用软件线
```

A. 硬件线

第一阶段：

```
现有 Pi 3B
+
XL4015 车载供电
+
现有 USBcan-II
```

完成：

- 车载稳定供电；
- USBcan连接；
- 实际车内运行验证。

同时只把 MT7628 等低成本主控作为后续参考，不阻塞软件。

B. OS / BSP 线

第一版不急于定制 OS。

先使用 Pi 上成熟 Linux 验证：

```
USBcan
Wi-Fi
uHTTPd
QuickJS
Service
Storage
```

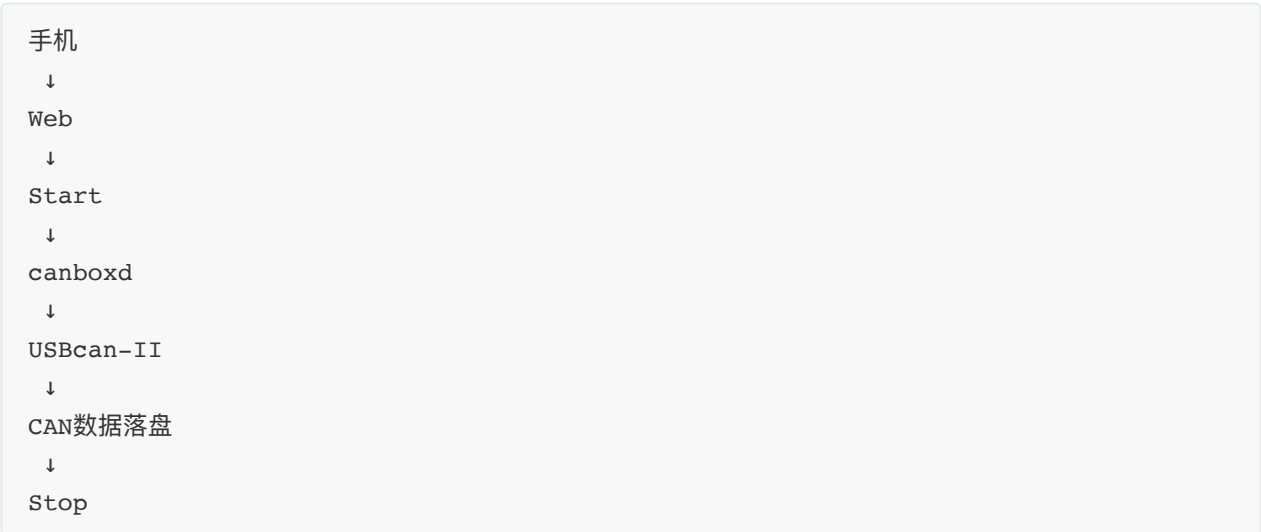
与此同时可以独立研究目标板：

```
OpenWrt / Buildroot
Boot
Kernel
Driver
Wi-Fi
USB
Image
```

等应用稳定后，再把它移植到目标硬件并裁成专用 OS。

C. 应用软件线

优先完成最短闭环：



第一版闭环跑通后，再逐项增加：

Case
Session
Round
滚动文件
掉电保护
AP / STA
时间同步
日志
上传
Recovery
OTA

4. 三线合流需要提前冻结的 Contract

只冻结真正影响并行开发的接口：

USBcan Adapter Interface
Device State Model
数据目录 / 文件格式
Case / Session / Round Metadata
配置格式
Web API

应用层不应该知道底下最终是 Pi 3B、MT7628 还是其他 Linux 主控。

5. 推荐实施顺序

需求冻结
↓
Pi 3B 第一版硬件环境
↓
最小软件闭环
Web → Start → CAN → File → Stop
↓
现场实际采集验证
↓
补齐设备管理能力
↓
并行验证目标低成本硬件 / OS
↓
应用稳定
↓
移植目标硬件
↓
定制专用 OS
↓
形成 CAN BOX V1

6. 当前停止点

本轮已经得到：

需求
+
硬件参考方案
+
软件架构
+
开发路线

因此当前没有必要继续扩展：

云平台
自研 CAN 硬件
手机 App
4G
CAN-FD
复杂 OTA
最终 PCB
极限 BOM

这些问题等 V0 真正撞到需求再处理。

五、一句话冻结

CAN BOX V0 是一个低成本、无屏、Linux 化的 CAN
智能采集网关：利用汽修店现有 USBcan-II，通过车载 12V
供电、本地可靠存储、Wi-Fi 和手机 Web，实现 Case 驱动的标准化 CAN
采集、管理、上传和后续远端诊断；第一版以 Pi 3B
快速验证完整闭环，硬件、OS、应用三线并行，成熟后再移植到低成本专用
Linux 主控。

阶段结论：本轮"脑子满汉全席"到此收桌。后续回到 V0 验证，不提前为 V2 解决问题。